

LECTURE

Introduction to the Terraform Workflow



The Terraform Workflow (Official)

1

Write

Create or Modify your Terraform Configuration Files

2

Plan

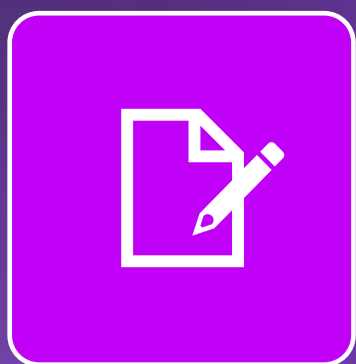
Preview your changes before applying them (aka dry run)

3

Apply

Implement the Changes According to your Configurations

Terraform Workflow



Write
Configuration
(HCL)



Initialize



Plan



Apply



Developing Terraform Configuration



What Does "Develop Configuration" Mean?

The Write stage involves creating one or more configuration files that define your desired state – the end result you want for your infrastructure.

Terraform configurations are iterative, meaning you can build and refine your configuration incrementally.



Defining Desired State

You describe the infrastructure components (e.g., VMs, Networks) and the properties for each. Terraform ensures that the "real-world" infrastructure matches this state



terraform init

`terraform init` is the first command you run in a new Terraform project to set up your working directory

- **What Does It Do?**

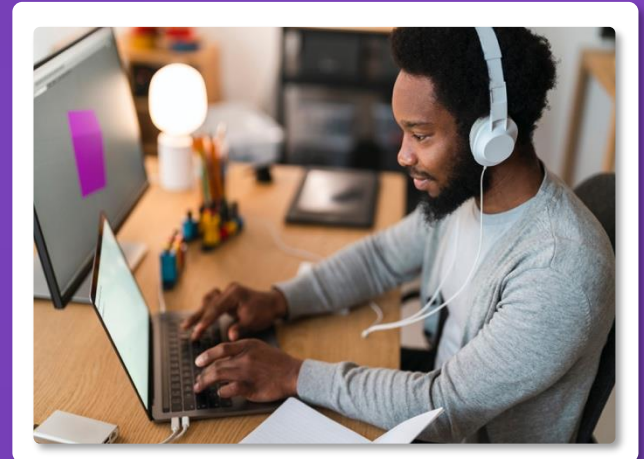
It prepares the environment to run other Terraform commands. Downloads the required providers (e.g., AWS, Azure, GCP) and modules used in the Terraform configuration.

- **Backend Management**

`terraform init` also configures the backend for storing Terraform state and creates the lock file

- **When Do You Use It?**

At the start of a new project or any time you add, remove, or upgrade providers/modules or change the backend settings in your configuration.





The Terraform Lock File

Every time you run `terraform init`, Terraform creates or updates a file called `.terraform.lock.hcl` in your working directory



Purpose

Locks dependency versions for consistency across your environments



File Name and Location

The name is `.terraform.lock.hcl` and it's stored in your working directory (same as `.tf` files)



Best Practices

Commit to Version Control and Managed Automatically by Terraform

terraform init



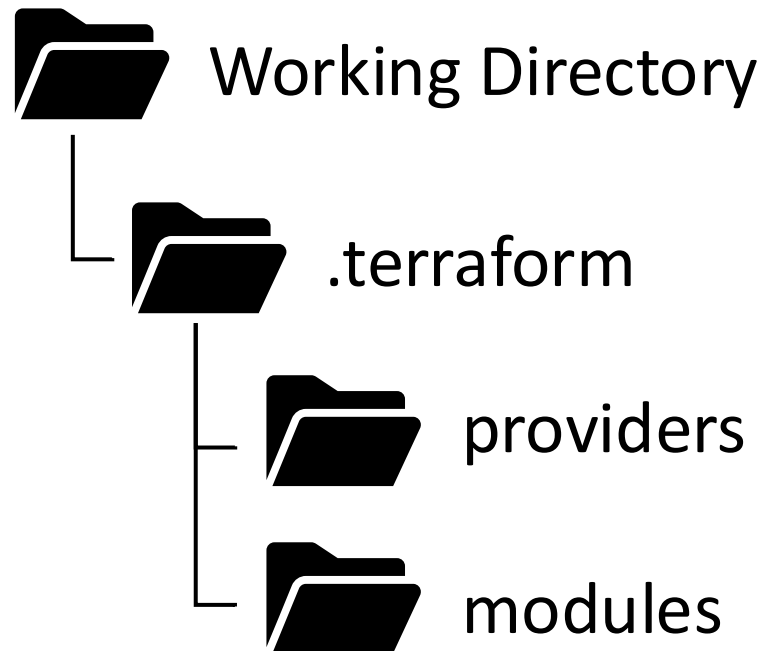
```

$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/vsphere...
- Finding latest version of hashicorp/random...
- Finding latest version of hashicorp/vault...
- Installing hashicorp/vault v4.5.0...
- Installed hashicorp/vault v4.5.0 (signed by HashiCorp)
- Installing hashicorp/vsphere v2.10.0...
- Installed hashicorp/vsphere v2.10.0 (signed by HashiCorp)
- Installing hashicorp/random v3.6.3...
- Installed hashicorp/random v3.6.3 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```



The `.terraform` Directory

Terraform uses the `.terraform` directory to store the project's providers and modules

- ✦ The `.terraform/providers` directory stores cached versions of the configuration's providers.
- ✦ The `.terraform/modules` directory contains downloaded versions of any referenced modules.



Working with the .terraform Directory

01

Should the `.terraform` directory be in version control?

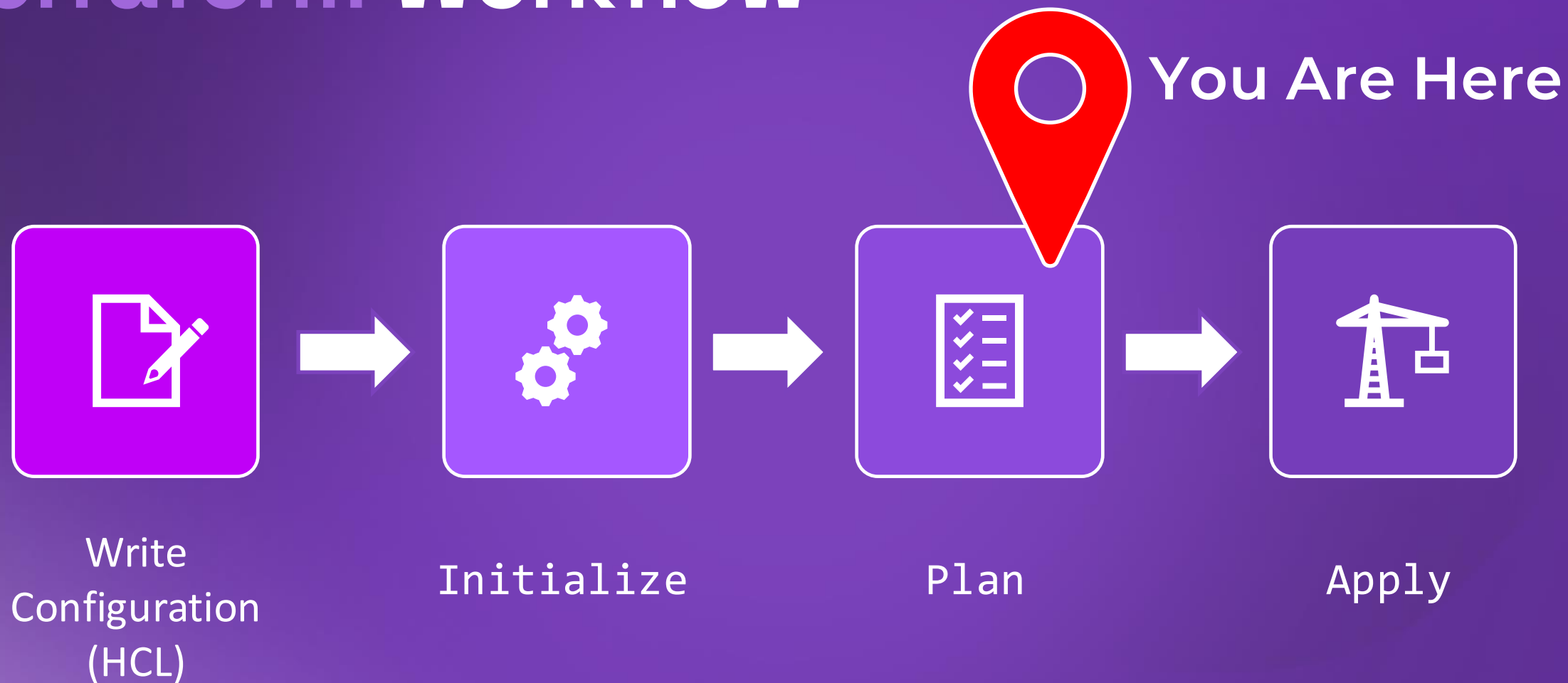
- ✧ No, add to your `.gitignore` file
- ✧ Contains easily recreatable files
- ✧ Can be large in size and contain platform-specific files

02

Can you delete the `.terraform` directory?

- ✧ Technically, yes
- ✧ Run `terraform init` to create it
- ✧ Useful for troubleshooting or cleaning up old dependencies

Terraform Workflow



terraform plan



terraform plan generates an execution plan showing the changes that Terraform will make to the 'real-world' resources in order to match the desired state configuration



What Does It Do?

Provides you with an output of the changes that will happen if you apply the changes. It shows you the resources to be created (+), updated (~), or destroyed (-).



Perform a Dry-Run of Your Changes

terraform plan gives you the ability to perform a 'dry-run' without impacting 'real-world' resources. It helps avoid unintended changes by showing exactly what Terraform will do



Why a Plan is Important

It is crucial for reviewing changes in production environments to prevent downtime out errors. changes,uld be run before applying any changes but it is **NOT** required for a **terraform apply**



terraform plan



TERMINAL

```
$ terraform plan
vault_generic_secret.example_secret: Refreshing state... [id=secret/example]
random_pet.example: Refreshing state... [id=smashing-mutt]
vault_generic_secret.example_kv: Refreshing state... [id=secret/example1]
```

Refreshes State

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

- + create
- ~ update in-place
- destroy
- /+ destroy and then create replacement

Terraform will perform the following actions:

```
# random_pet.additional_pet will be created
+ resource "random_pet" "additional_pet" {
  + id          = (known after apply)
  + length      = 3
  + separator   = "_"
}

# random_pet.example must be replaced
-/+ resource "random_pet" "example" {
  ~ id          = "smashing-mutt" -> (known after apply)
  ~ length      = 2 -> 3 # forces replacement
  # (1 unchanged attribute hidden)
}
```

Create new resource

Replaced

terraform plan



TERMINAL

...continued from previous output

```
# vault_generic_secret.example_kv will be updated in-place
~ resource "vault_generic_secret" "example_kv" {
  ~ data_json      = (sensitive value)
    id              = "secret/example1"
  # (4 unchanged attributes hidden)
}
```

Modify

```
# vault_generic_secret.example_secret will be destroyed
# (because vault_generic_secret.example_secret is not in configuration)
- resource "vault_generic_secret" "example_secret" {
  - data              = (sensitive value) -> null
  - data_json         = (sensitive value) -> null
  - delete_all_versions = false -> null
  - disable_read       = false -> null
  - id                 = "secret/example" -> null
  - path              = "secret/example" -> null
}
```

destroy

Plan: 2 to add, 1 to change, 2 to destroy.

Overall plan

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

Saving a Plan

Guaranteeing Execution for Review and Approval Workflows



Benefits of Saving Plans

The exact actions in the saved plan will be executed, no surprises



Review & Approval Workflows

Save plan, have it reviewed, then apply later with confidence



How to Save a Plan

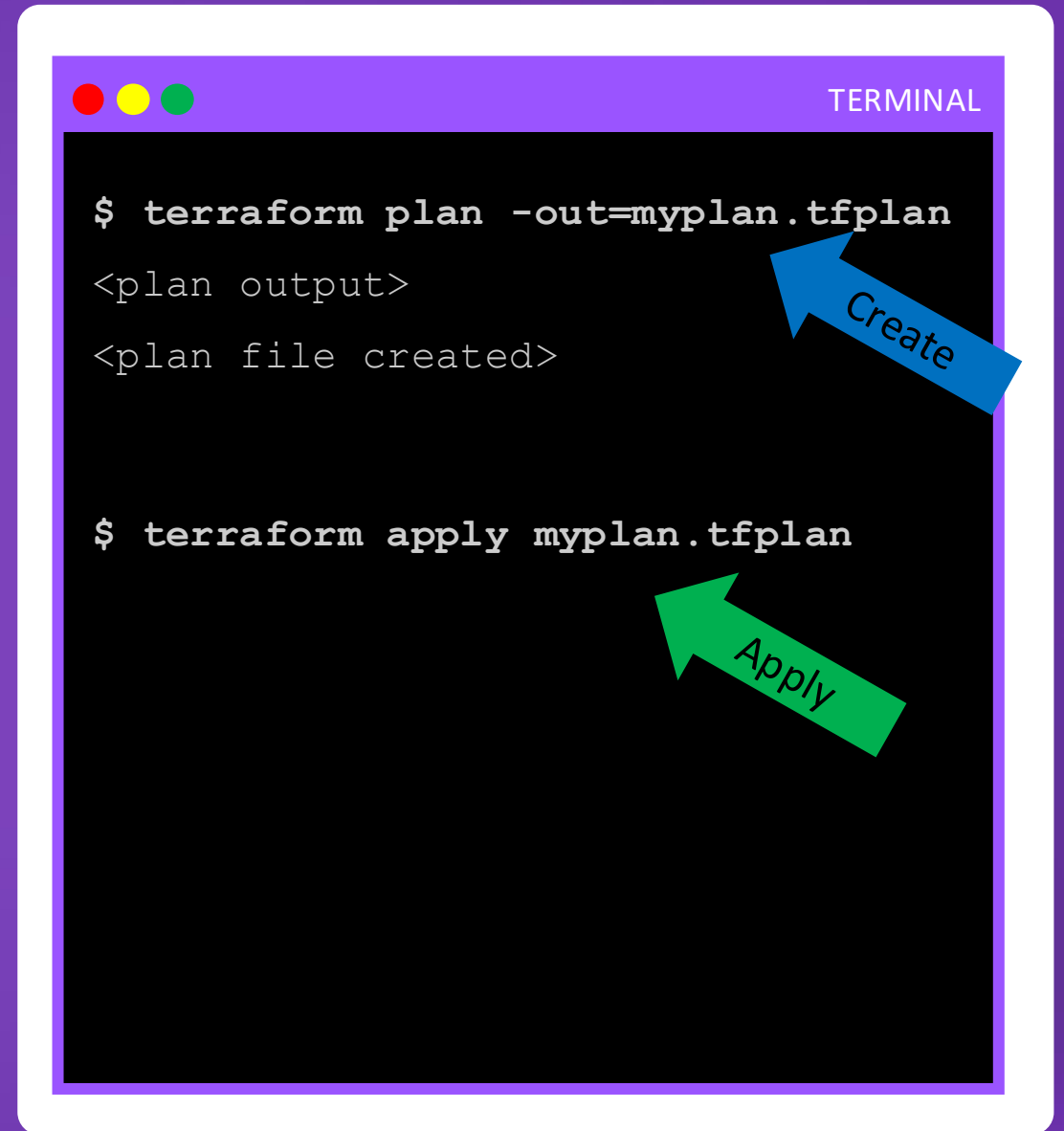
```
$ terraform plan -out=myplan.tfplan
```

(file name can be anything you choose)



How to Apply a Saved Plan

```
$ terraform apply myplan.tfplan
```





Resource Graph

So what actually happens when you run a Terraform plan?

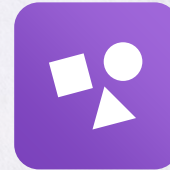
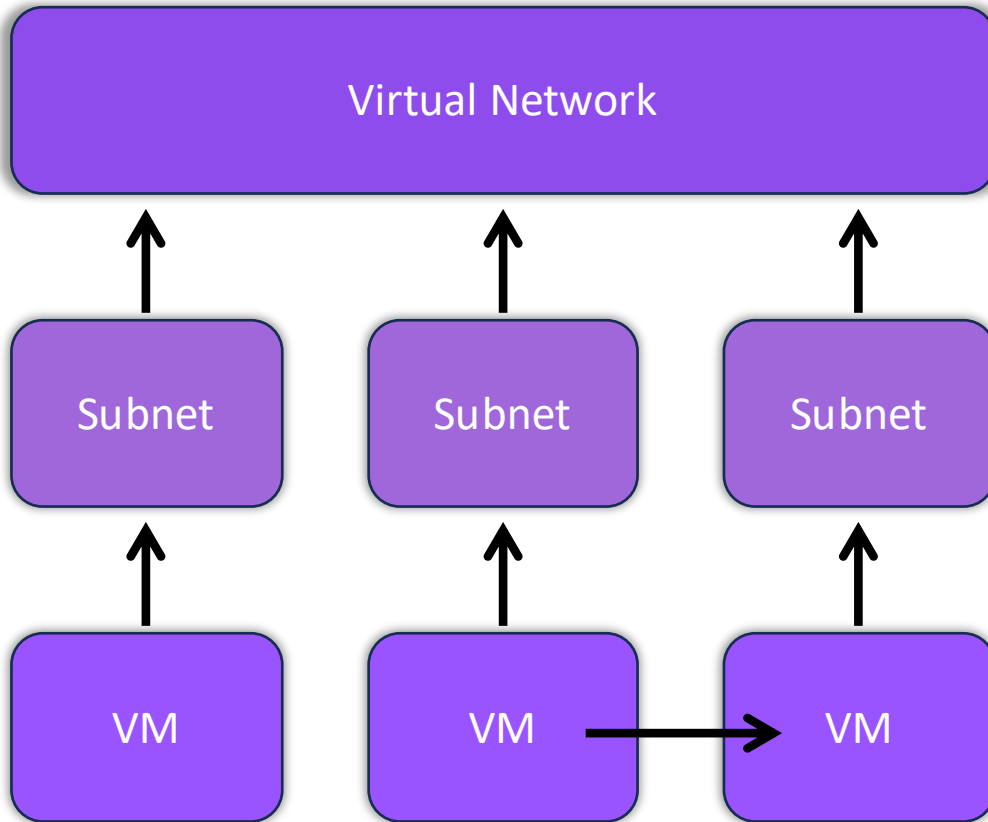
Terraform automatically builds a dependency graph from your configuration to determine the order in which to create, update, or destroy resources.

- ✧ Resource order in your .tf files doesn't matter
- ✧ Resource graph is how Terraform achieves parallel execution
- ✧ Foundation for understanding resource dependencies



Key Concept

The graph is built from resource references - when one resource refers to another's attributes, Terraform automatically creates a dependency.



Dependencies

Two types of dependencies:

- ✧ **Implicit Dependencies (automatic)** – a reference to another resource that Terraform already knows about
- ✧ **Explicit Dependencies (Manual)** – Used when there's a dependency that Terraform can't automatically detect from resource references alone. Use `depends_on` for create an explicit dependency

Parallel Resource Operations

How Terraform Walks the Graph



Terraform Processes the Graph with Parallel Execution

Processes nodes as soon as their dependencies are satisfied. Non-dependent resources are created/modified simultaneously



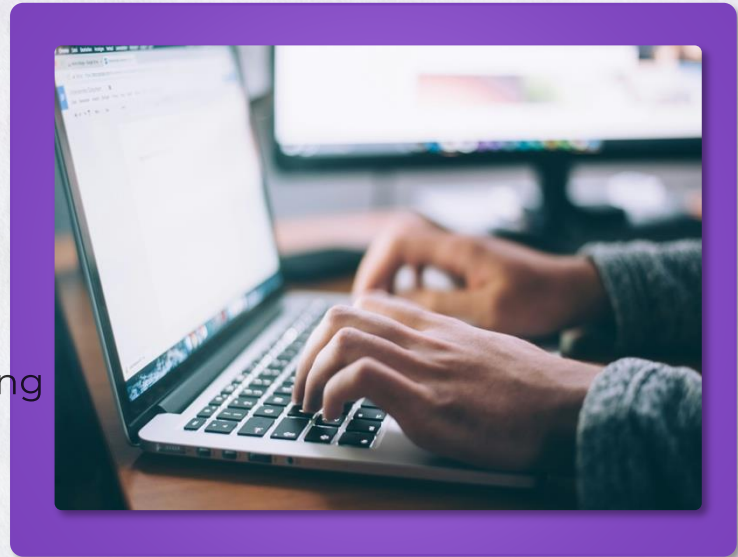
Default Configuration

By default, Terraform manages **10** operations in parallel but can be modified using the `-parallelism` flag (`terraform apply -parallelism=5`)



When to Change Parallelism

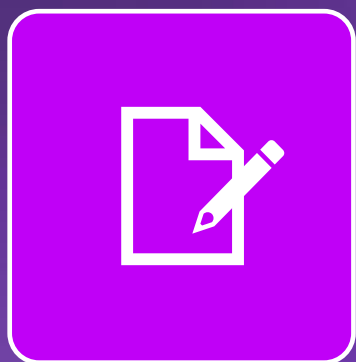
If you're running into API rate limiting issues, or for debugging/troubleshooting purposes. Most users never need to change this setting.



Terraform Workflow



You Are Here



Write
Configuration
(HCL)



Initialize



Plan



Apply



terraform apply

terraform apply tells Terraform to provision real infrastructure

- **What Does It Do?**

It takes your configuration and creates the resources as defined in your configuration. This is where Terraform makes changes to your infrastructure.

- **Refreshes State**

terraform apply runs a plan first to compare your existing infrastructure to your desired state. By default, Terraform will prompt you to proceed with the proposed changes.

- **When Do You Use It?**

Anytime you want to make changes to your infrastructure. This includes adding, modifying, or destroying real-world resources.





Understanding Apply Operations

Understanding what happens behind the scenes when Terraform modifies your infrastructure



State Locking

Terraform locks the state file during apply to prevent concurrent modifications.



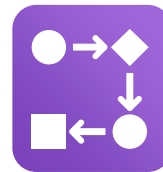
State Updates

State file is updated after each resource is successfully created/modified/destroyed.



No Automatic Rollback

If a resource fails during apply, Terraform does not rollback previous successful changes. They remain in place.



Respects Dependencies

Commit to Version Control and Managed Automatically by Terraform.

terraform apply



Standard Apply →

```

$ terraform apply
Confirm (Y/N) :

```

Apply a Saved Plan →

```

$ terraform apply myplan.tfplan

```

Apply & Auto Approve →

```

$ terraform apply -auto-approve
(skips confirmation prompt - use carefully)

```

-input=false is also helpful for automation/CI/CD pipelines

Best Practices for terraform apply



1

Before You Apply

- ✓ Always review the plan
- ✓ Ensure You're in the Right Directory
- ✓ Test in Non-Production

2

During Apply

- ✓ Read the Plan Carefully
- ✓ Watch for Unexpected Changes
- ✓ Be Patient with Large Applies

3

After You Apply

- ✓ Verify the Changes
- ✓ Commit Your Configuration
- ✓ Document Significant Changes



terraform destroy



terraform destroy terminates resources managed by Terraform in the current workspace or working directory.



Using the Destroy Command

Explicitly destroys all infrastructure being managed by Terraform. This operation does prompt for confirmation before proceeding. Shows destruction plan first.



Removing Individual Resources from your Configuration

Delete resource blocks from your .tf files and then run **terraform apply**. Terraform detects removed resources and destroys the real-world resource.



State File Updates

Once a resource is destroyed, Terraform removes it from the state file as well since it is no longer being managed.

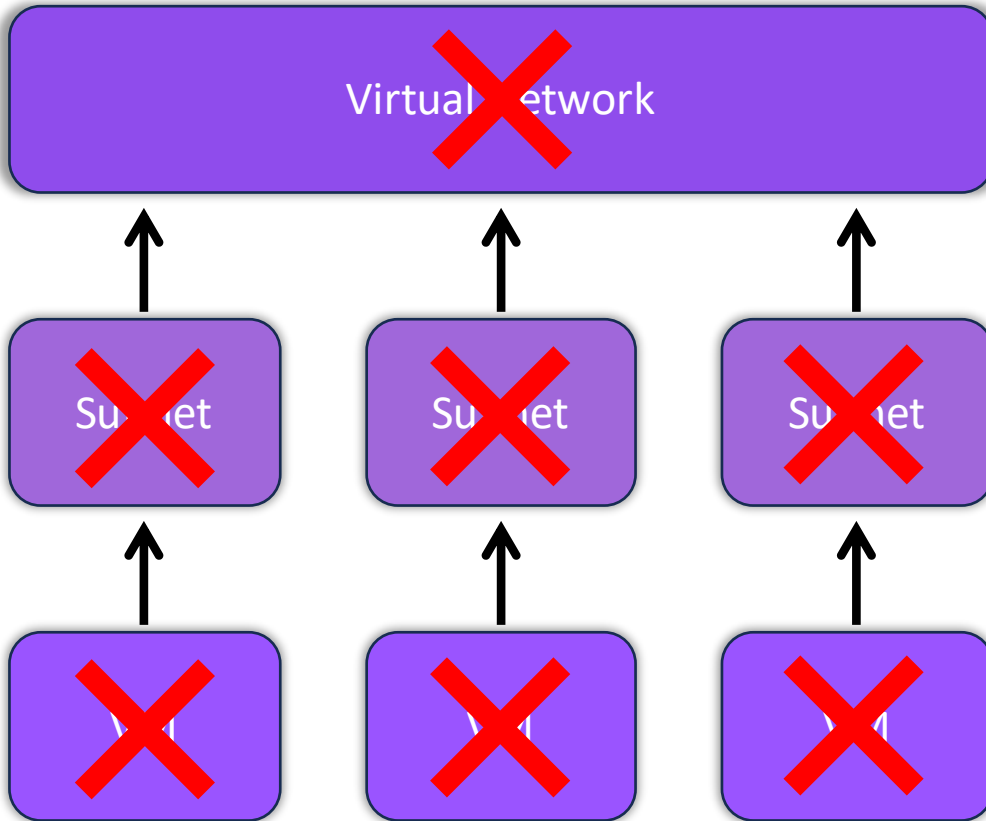




WARNING!

Warning: Destroy operations are irreversible.

Always review the plan carefully



Destroying Resources

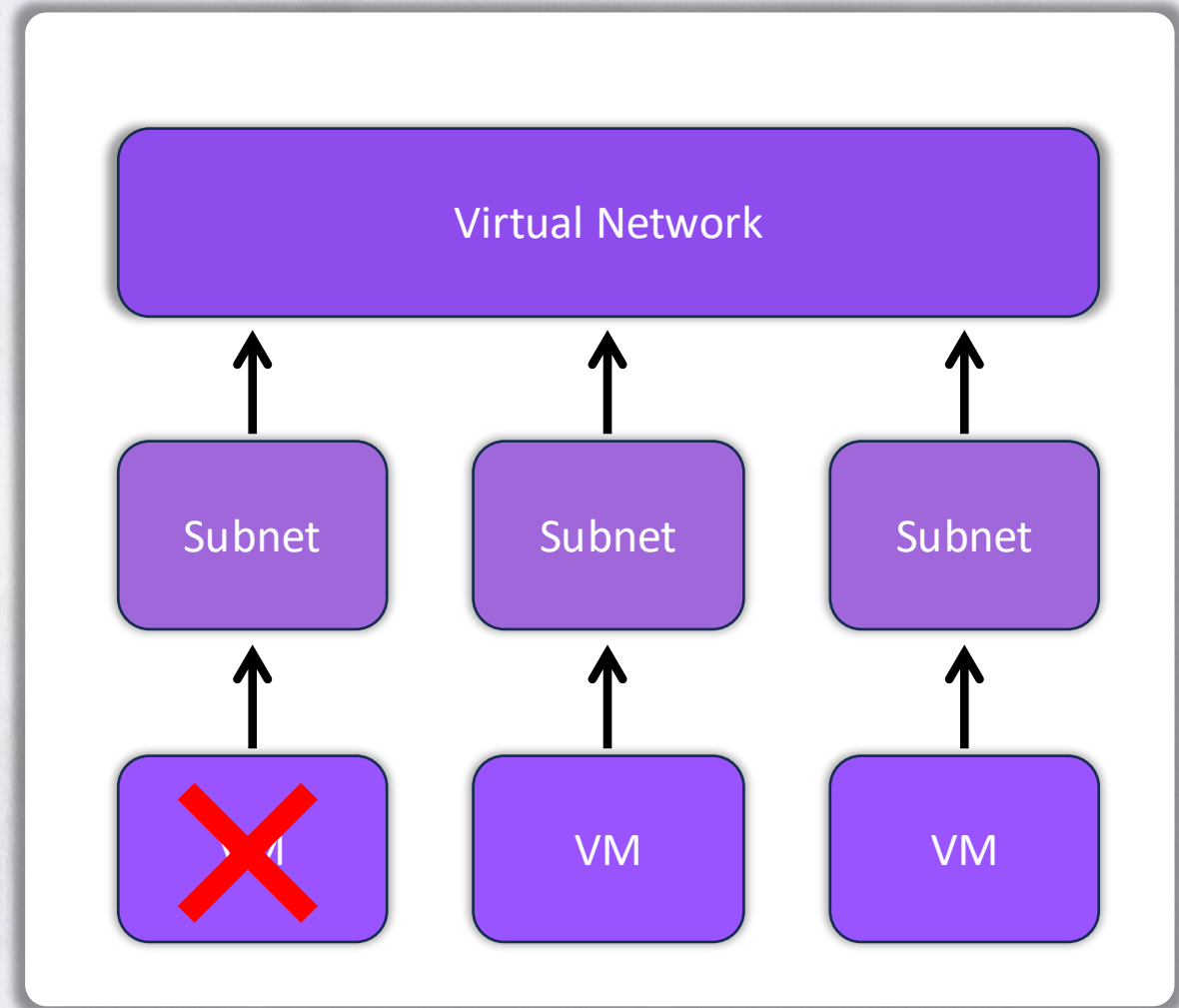
The default `terraform destroy` command will destroy all resources being managing by Terraform.



Destroying a Single Resource

You can target specific resources, if needed using the destroy command.

```
terraform destroy -target=<resource_name>
```



Terraform Validate



Validate Your Configurations

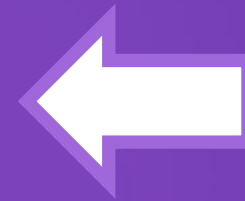
The `terraform validate` command checks your Terraform configuration files for syntax errors and ensures they are structurally valid. It does not check whether the configuration will successfully deploy but ensures your code is properly written and references resources and variables correctly.

This is an essential step to catch errors early before running commands like `plan` or `apply`.



Technical Details of Validate

```
1 resource "aws_instance" "example" {  
2     ami          = "ami-12345678"  
3     instance_type = "t2.micro"  
4     tags = {  
5         Name = "example"  
6     }  
7 }
```



terraform validate

Does Not Communicate with
Providers

Because of That, Validate
Doesn't Catch Every Error



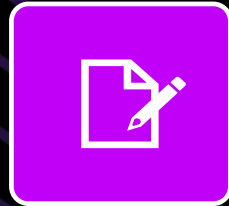
Requirements

Initialize



`terraform
init`

Validate



`terraform
validate`

Plan



`terraform
plan`

Apply



`terraform
apply`

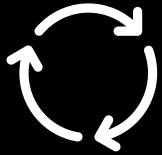
Destroy



`terraform
destroy`

You MUST initialize your working directory first before running `terraform validate`

SUMMARY



Terraform workflow begins with `terraform init`, which sets up the project and downloads necessary providers and modules.



`terraform plan` generates an execution plan, allowing you to preview changes before applying them



`terraform apply` is then used to make the planned changes to your infrastructure.



By default, Terraform will ask for confirmation before making any changes to real-world infrastructure



Each step in the workflow is crucial for managing infrastructure predictably and efficiently.



Following this workflow ensures consistency and control over infrastructure changes.